



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Final Lab Report

Course : Software Engineering Lab

Course Code : CSE 3108

Submission Date : 22/09/2025

Submitted by	Submitted to
Adnan Zaman Niloy, Student ID: 210142 Year: 3 rd Semester: 1 st Session: 2021-22 Program: Undergraduate. Department of Computer Science and Engineering, Jashore University of Science and Technology	Dr. Md. Nasim Adnan, Assistant Professor, Department of Computer Science and Engineering, Jashore University of Science and Technology

1.Introduction

This lab focused on modeling and implementing software systems using UML and Object-Oriented Programming (OOP) concepts. UML diagrams (Class, Use Case, Activity, and Sequence) were created to capture system structure and behavior, while coding tasks in Java applied principles like polymorphism, abstraction, and design patterns (Adapter, Builder) to ensure modularity and reusability.

2. Objectives

- To analyze and understand the requirements of the system.
- To design and evaluate structural and behavioral UML diagrams.
- To apply Object-Oriented Programming concepts such as polymorphism, abstraction, navigation, and downcasting.
- To implement and demonstrate the use of common software design patterns (Adapter and Builder).
- To understand how different components of the system interact with each other and contribute to modular, reusable, and maintainable design.

3.1 Diagram Tasks

Task: 01 Use Case Diagram

- **Actors:** Student, Advertiser, Maintenance Officer.
- **Use Cases:** Register, Login/Logout, Update Test Paper, Request Paper, Submit Query, View FAQ, Make Ad Request, Process Ad Request, Handle Invalid Payment, Process Queries, Update Records, Include Process Test.
- **Explanation:** Shows high-level interactions. Students use academic features, advertisers submit ads, while the officer validates and manages processes.

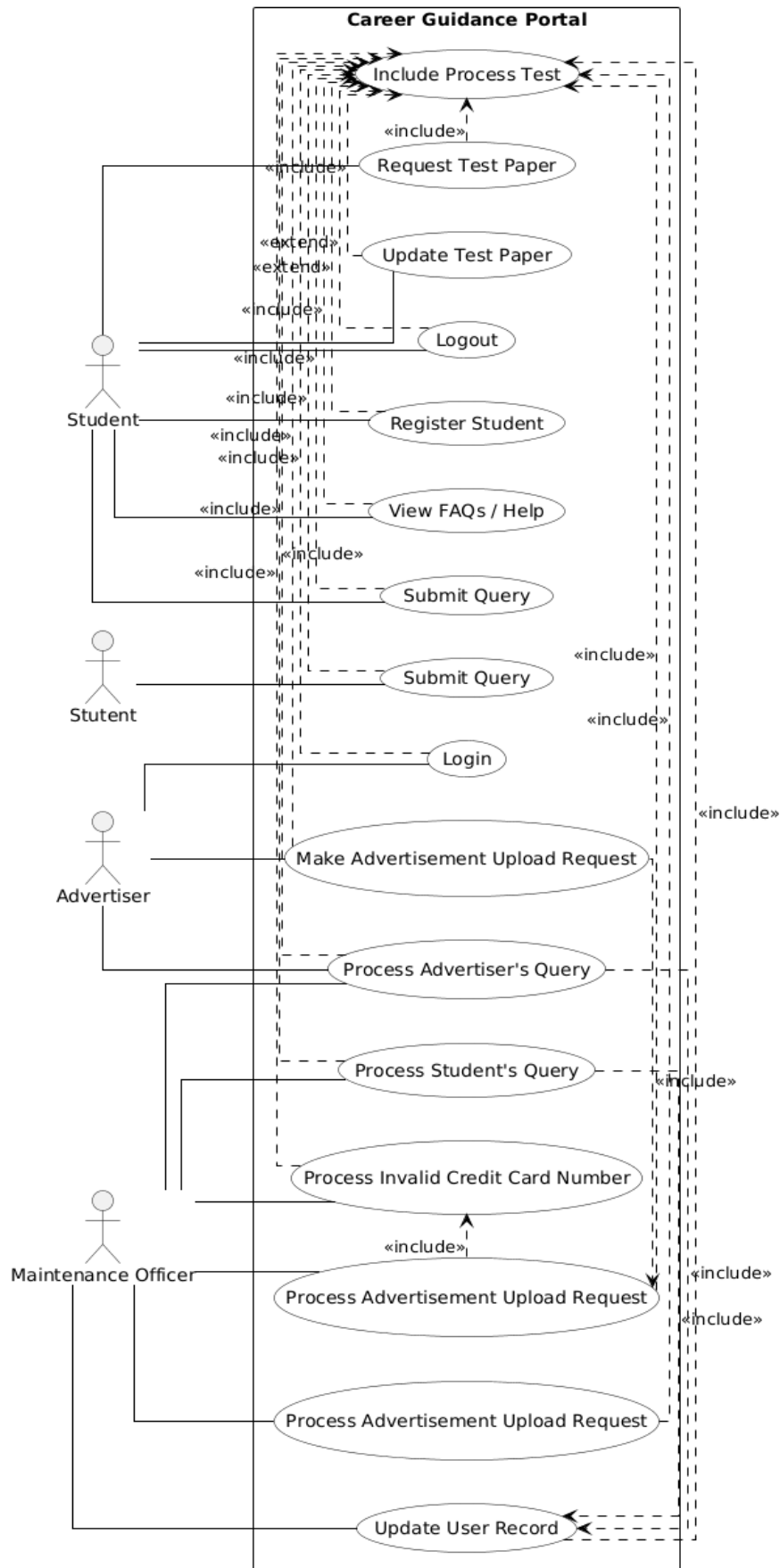


Fig:01

Task:02 Activity Diagram

- **Student:** Register → Login → Request Paper → Pay → Receive Key → Submit Answers → Get Summary → (Optional) Submit Query.
- **Advertiser:** Register → Login → Submit Ad → Admin Approval → Pay → Ad Uploaded.
- **Officer:** Validates payments, approves ads/papers, resolves queries.
- **Explanation:** Captures sequential/conditional flows like payment validation, query handling, and ad approval.

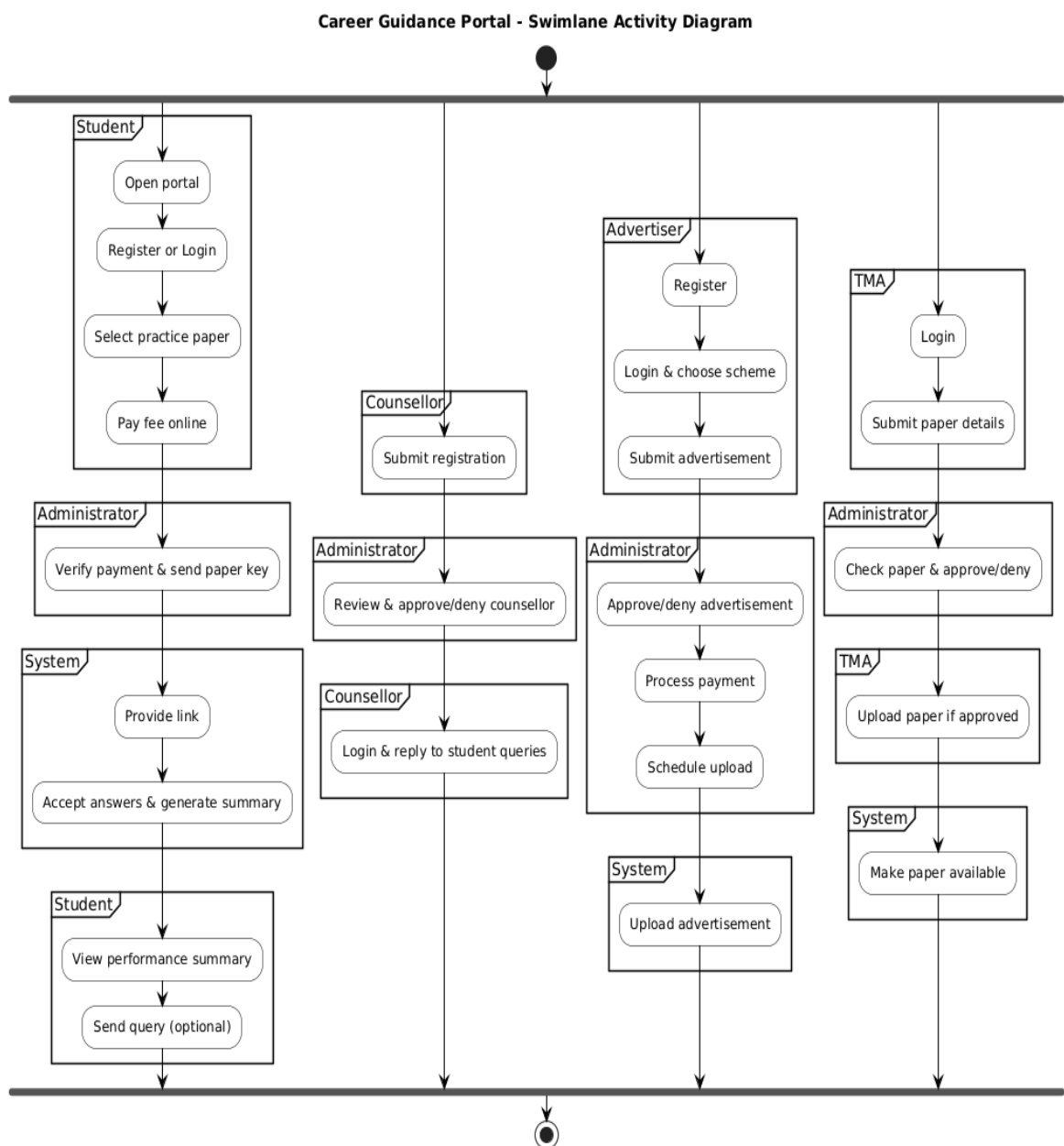


Fig:02

Task 03: Class Diagram

- **Main Classes:**

1. Student (register(), login(), requestTestPaper(), submitQuery()).
2. Advertiser (login(), makeAdRequest()).
3. MaintenanceOfficer (processQueries(), updateUserRecord(), approveAds()).
4. TestPaper (update(), evaluateSubmission(), generateSummary()).
5. Advertisement (approve(), scheduleUpload()).

- **Relationships:** User → (Student, Advertiser, Officer); Officer manages TestPapers & Ads; Student depends on Officer for support.
- **Explanation:** Shows static structure, attributes, and OOP relationships.

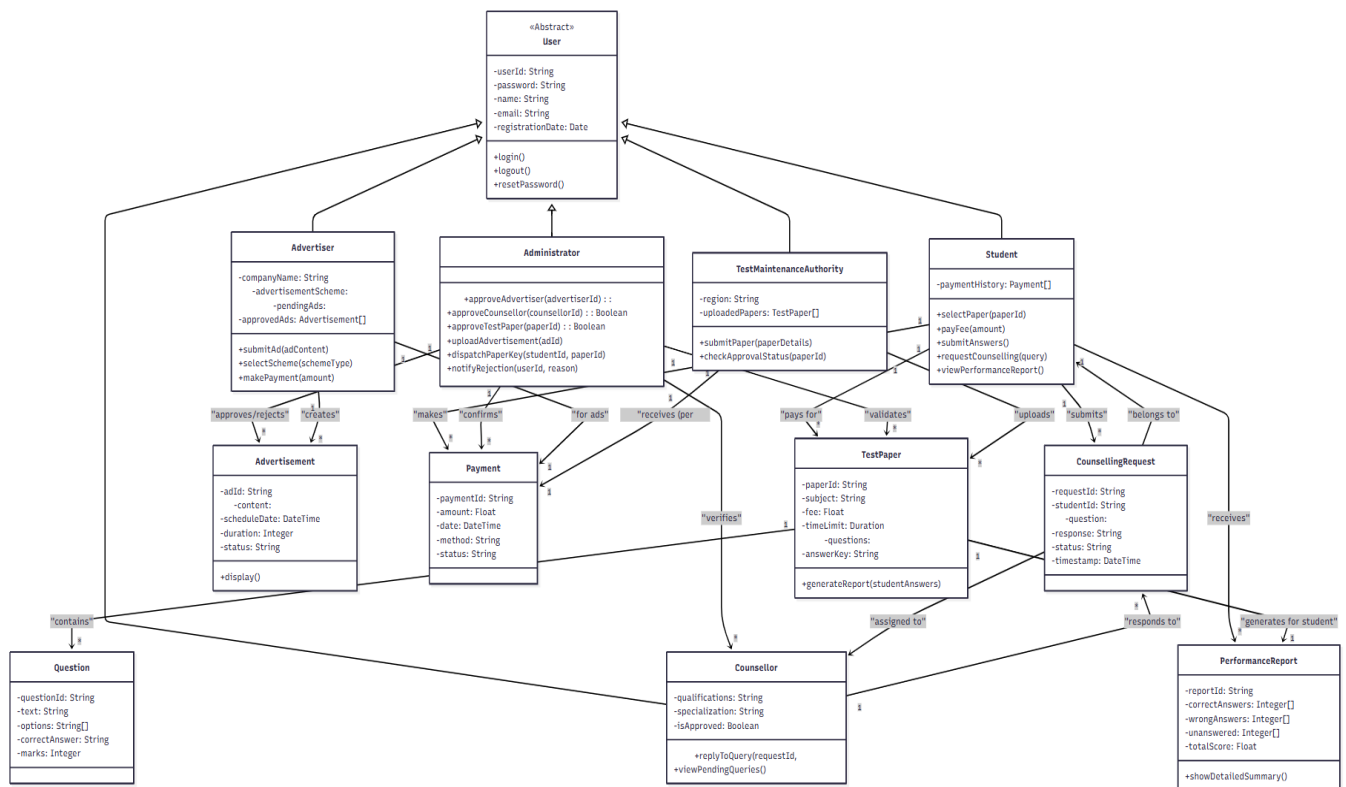


Fig:03

4.2: Coding Task

Task 1: Polymorphism

Polymorphism enables one interface to represent multiple implementations. In C#, overriding virtual methods allows behavior to differ based on the runtime object.

Procedure

- Defined base class clsEmployee with virtual method WriteInfo().
- Created subclasses clsTeacher and clsOfficer overriding the method.
- Stored them in an employee array and invoked WriteInfo() uniformly.

Implementation:

using System;

namespace Polymorphism

```
{
    class Program
    {
        public class clsEmployee
        {
            public string Name { get; set; }
            public string Designation { get; set; }

            public virtual void WriteInfo()
            {
                Console.WriteLine("Employee Info:");
                Console.WriteLine("Name:      " + this.Name);
                Console.WriteLine("Designation: " + this.Designation);
            }
        }

        public class clsTeacher : clsEmployee
        {
            public string Department { get; set; }

            public override void WriteInfo()
            {
                Console.WriteLine("Teacher Info:");
                Console.WriteLine("Name:      " + this.Name);
                Console.WriteLine("Designation: " + this.Designation);
                Console.WriteLine("Department: " + this.Department);
            }
        }

        public class clsOfficer : clsEmployee
```

```

{
    public string Office { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Officer Info:");
        Console.WriteLine("Name:      " + this.Name);
        Console.WriteLine("Designation: " + this.Designation);
        Console.WriteLine("Office:    " + this.Office);
    }
}

static void Main(string[] args)
{
    clsEmployee[] oEmployees = new clsEmployee[3];

    clsTeacher oTeacher = new clsTeacher();
    oTeacher.Name = "Nasim";
    oTeacher.Designation = "Assistant Director";
    oTeacher.Department = "IT OCD";
    oEmployees[0] = oTeacher;

    clsOfficer oOfficer = new clsOfficer();
    oOfficer.Name = "Ahdab";
    oOfficer.Designation = "Clark (Grade-1)";
    oOfficer.Office = "HR";
    oEmployees[1] = oOfficer;

    clsEmployee oEmployee = new clsEmployee();
    oEmployee.Name = "Abdullah";
    oEmployee.Designation = "Clark (Grade-3)";
    oEmployees[2] = oEmployee;

    for (int i = 0; i < oEmployees.Length; i++)
    {
        oEmployees[i].WriteInfo();
        Console.WriteLine();
    }

    Console.ReadLine();
}
}
}

```

Output:

Teacher Info:

Name: Nasim

Designation: Assistant Director

Department: ITOCD

Officer Info:

Name: Ahdab

Designation: Clark (Grade-1)

Office: HR

Employee Info:

Name: Abdullah

Designation: Clark (Grade-3)

Analysis

Outputs confirmed role-specific details for teachers, officers, and generic employees from the same method call.

Design Impact

This design makes the system extensible. New roles can be added without changing call sites, following the Open/Closed Principle.

Task 2: Polymorphism More

Polymorphism scales to larger hierarchies, maintaining flexibility when more subtypes are added.

Procedure

- Added clsStaff as another subclass of clsEmployee.
- Used the same base array to invoke WriteInfo() polymorphically.

Implementation

```
using System;
namespace Polymorphism
{
    class Program
    {
        // Base class
        public class clsEmployee
        {
```



```

public string Name { get; set; }
public string Designation { get; set; }

public virtual void WriteInfo()
{
    Console.WriteLine("Employee Info:");
    Console.WriteLine($"Name: {Name}");
    Console.WriteLine($"Designation: {Designation}");
}
}

// Derived class Teacher
public class clsTeacher : clsEmployee
{
    public string Department { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Teacher Info:");
        Console.WriteLine($"Name: {Name}");
        Console.WriteLine($"Designation: {Designation}");
        Console.WriteLine($"Department: {Department}");
    }
}

// Derived class Officer
public class clsOfficer : clsEmployee
{
    public string Office { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Officer Info:");
        Console.WriteLine($"Name: {Name}");
        Console.WriteLine($"Designation: {Designation}");
        Console.WriteLine($"Office: {Office}");
    }
}

// Derived class Staff
public class clsStaff : clsEmployee
{
    public double OverTime { get; set; }

```

```

public override void WriteInfo()
{
    Console.WriteLine("Staff Info:");
    Console.WriteLine($"Name: {Name}");
    Console.WriteLine($"Designation: {Designation}");
    Console.WriteLine($"Overtime: {OverTime}");
}
}

// Main method
static void Main(string[] args)
{
    clsEmployee[] oEmployees = new clsEmployee[3];

    clsTeacher oTeacher = new clsTeacher()
    {
        Name = "Nasim",
        Designation = "Assistant Director",
        Department = "IT OCD"
    };
    oEmployees[0] = oTeacher;

    clsOfficer oOfficer = new clsOfficer()
    {
        Name = "Ahdab",
        Designation = "Clark (Grade-1)",
        Office = "HR"
    };
    oEmployees[1] = oOfficer;

    clsStaff oStaff = new clsStaff()
    {
        Name = "Kuddus",
        Designation = "Sweeper",
        OverTime = 10.75
    };
    oEmployees[2] = oStaff;

    // Polymorphism in action
    foreach (clsEmployee emp in oEmployees)
    {
        emp.WriteInfo();
    }
}

```

```

        Console.WriteLine();
    }

    Console.ReadLine();
}
}
}

```

Output

Teacher Info:

Name: Nasim

Designation: Assistant Director

Department: ITOCD

Officer Info:

Name: Ahdab

Designation: Clerk (Grade-1)

Office: HR

Staff Info:

Name: Kuddus

Designation: Sweeper

Overtime: 10.75

Analysis

Console results included teacher, officer, and staff-specific outputs, each responding correctly to the override.

Design Impact

Expanding hierarchies does not break existing code. Polymorphism ensures decoupling between callers and implementation details.

Task 3: Polymorphism with Downcasting

Downcasting converts a base class reference back into a derived class reference, granting access to subtype-specific members.

Procedure

- Declared clsEmployee references pointing to clsTeacher.
- Called WriteInfo() polymorphically.
- Downcasted to clsTeacher for direct access to Papers.

Implementation

```

using System;
namespace Polymorphism
{
    class Program
    {
        public class clsEmployee    // Base class
        {
            public string Name { get; set; }
            public string Designation { get; set; }

            public clsEmployee() { Name = ""; Designation = ""; }

            public virtual void WriteInfo()
            {
                Console.WriteLine("Employee Info:");
                Console.WriteLine("Name: " + this.Name);
                Console.WriteLine("Designation: " + this.Designation);
            }
        }

        public class clsTeacher : clsEmployee // Derived class
        {
            public string Department { get; set; }
            public int Papers { get; set; }

            public clsTeacher() : base() { Department = ""; Papers = 0; }

            public override void WriteInfo()
            {
                Console.WriteLine("Teacher Info:");
                Console.WriteLine("Name: " + this.Name);
                Console.WriteLine("Designation: " + this.Designation);
                Console.WriteLine("Department: " + this.Department);
                Console.WriteLine("Papers: " + this.Papers);
            }
        }

        static void Main(string[] args)
        {
            clsEmployee[] oEmployees = new clsEmployee[2];

            clsTeacher oTeacher1 = new clsTeacher();
            oTeacher1.Name = "Nasim";

```

```

oTeacher1.Designation = "Assistant Professor";
oTeacher1.Department = "CSE";
oTeacher1.Papers = 20;
oEmployees[0] = oTeacher1;

clsTeacher oTeacher2 = new clsTeacher();
oTeacher2.Name = "Abdullah";
oTeacher2.Designation = "Lecturer";
oTeacher2.Department = "EEE";
oTeacher2.Papers = 5;
oEmployees[1] = oTeacher2;

// Polymorphism
for (int i = 0; i < 2; i++)
{
    oEmployees[i].WriteInfo();
    Console.WriteLine();
}

// Downcasting
for (int i = 0; i < 2; i++)
{
    clsTeacher oTeacherDC = (clsTeacher)oEmployees[i];
    oTeacherDC.WriteInfo();
    Console.WriteLine();
}

Console.ReadLine();
}
}
}

```

Result/Output:

Teacher Info:

Name: Nasim

Designation: Assistant Professor

Department: CSE

Papers: 20

Teacher Info:

Name: Abdullah

Designation: Lecturer

Department: EEE

Papers: 5

Teacher Info:

Name: Nasim

Designation: Assistant Professor

Department: CSE

Papers: 20

Teacher Info:

Name: Abdullah

Designation: Lecturer

Department: EEE

Papers: 5

Analysis

Downcasting allowed teacher-specific details to be displayed, but also revealed the risk of runtime errors if casts are invalid.

Design Impact

Downcasting reduces type safety and should be avoided unless necessary. Instead, exposing subtype behaviors via abstract or virtual methods is safer.

Task 4: Abstraction

Abstraction defines common templates while hiding implementation details. Abstract classes enforce consistent method definitions across subclasses.

Procedure

- Converted clsEmployee into an abstract class.
- Required all subclasses to implement WriteInfo().

Implementation:

```
using System;
namespace Polymorphism
{
    class Program
    {
        public abstract class clsEmployee
        {
            public string Name { get; set; }
        }
    }
}
```

```

    public string Designation { get; set; }

    public abstract void WriteInfo(); // abstract method
}

public class clsTeacher : clsEmployee
{
    public string Department { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Teacher Info:");
        Console.WriteLine("Name:      " + this.Name);
        Console.WriteLine("Designation: " + this.Designation);
        Console.WriteLine("Department: " + this.Department);
    }
}

public class clsOfficer : clsEmployee
{
    public string Office { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Officer Info:");
        Console.WriteLine("Name:      " + this.Name);
        Console.WriteLine("Designation: " + this.Designation);
        Console.WriteLine("Office:    " + this.Office);
    }
}

public class clsStaff : clsEmployee
{
    public double OverTime { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Staff Info:");
        Console.WriteLine("Name:      " + this.Name);
        Console.WriteLine("Designation: " + this.Designation);
        Console.WriteLine("Overtime:   " + this.OverTime);
    }
}

```

```

static void Main(string[] args)
{
    clsEmployee[] oEmployees = new clsEmployee[3];

    clsTeacher oTeacher = new clsTeacher
    {
        Name = "Nasim",
        Designation = "Assistant Director",
        Department = "IT OCD"
    };
    oEmployees[0] = oTeacher;

    clsOfficer oOfficer = new clsOfficer
    {
        Name = "Ahdab",
        Designation = "Clerk (Grade-1)",
        Office = "HR"
    };
    oEmployees[1] = oOfficer;

    clsStaff oStaff = new clsStaff
    {
        Name = "Kuddus",
        Designation = "Sweeper",
        OverTime = 10.75
    };
    oEmployees[2] = oStaff;

    for (int i = 0; i < 3; i++)
    {
        oEmployees[i].WriteInfo();
        Console.WriteLine();
    }

    Console.ReadLine();
}
}

```

Output:

Teacher Info:

Name: Nasim
 Designation: Assistant Director
 Department: ITOCD

Officer Info:

Name: Ahdab
 Designation: Clerk (Grade-1)
 Office: HR

Staff Info:

Name: Kuddus
 Designation: Sweeper
 Overtime: 10.75

Analysis

All subclasses produced consistent outputs, with enforced implementation of the abstract method.

Design Impact

Abstract base classes improve consistency across hierarchies, reduce accidental omissions, and support contract-driven design.

Task 5: Navigation

Object navigation allows one object to access another, but direct lookup often embeds logic tightly within classes.

Procedure

- Linked clsTeacher with clsDepartment through a property.
- Department ID was mapped directly to department names.

Implementation

```
using System;
namespace Polymorphism
{
    class Program
    {
        public class clsDepartment
        {
            public int DepartmentID { get; set; }
            public string DepartmentName { get; set; }
        }

        public abstract class clsEmployee
```

```

{
    public string Name { get; set; }
    public string Designation { get; set; }
    public abstract void WriteInfo();
}

public class clsTeacher : clsEmployee
{
    public int DepartmentID { get; set; }
    public int Papers { get; set; }

    public clsDepartment Department
    {
        get
        {
            var d = new clsDepartment();
            if (DepartmentID == 1) { d.DepartmentID = 1; d.DepartmentName = "CSE"; }
            else if (DepartmentID == 2) { d.DepartmentID = 2; d.DepartmentName =
"EEE"; }
            return d;
        }
    }

    public override void WriteInfo()
    {
        Console.WriteLine("Teacher Info:");
        Console.WriteLine("Name:      " + Name);
        Console.WriteLine("Designation: " + Designation);
        Console.WriteLine("Department: " + Department.DepartmentName);
        Console.WriteLine("Papers:    " + Papers);
    }
}

public class clsOfficer : clsEmployee
{
    public string Office { get; set; }
    public bool AssociationMember { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Officer Info:");
        Console.WriteLine("Name:      " + Name);
        Console.WriteLine("Designation: " + Designation);
    }
}

```

```

        Console.WriteLine("Office:    " + Office);
        Console.WriteLine("Association: " + AssociationMember);
    }
}

public class clsStaff : clsEmployee
{
    public double OverTime { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Staff Info:");
        Console.WriteLine("Name:      " + Name);
        Console.WriteLine("Designation: " + Designation);
        Console.WriteLine("Overtime:  " + OverTime);
    }
}

static void Main(string[] args)
{
    clsEmployee[] oEmployees = new clsEmployee[4];

    oEmployees[0] = new clsTeacher { Name = "Nasim", Designation = "Assistant
Director", DepartmentID = 1, Papers = 2 };
    oEmployees[1] = new clsTeacher { Name = "Nasim", Designation = "Assistant
Director", DepartmentID = 2, Papers = 2 };
    oEmployees[2] = new clsOfficer { Name = "Ahdab", Designation = "Clark (Grade-
1)", Office = "HR", AssociationMember = true };
    oEmployees[3] = new clsStaff { Name = "Kuddus", Designation = "Sweeper",
OverTime = 10.75 };

    for (int i = 0; i < oEmployees.Length; i++)
    {
        oEmployees[i].WriteInfo();
        Console.WriteLine();
    }

    Console.ReadLine();
}
}

```

Result/Output:

Teacher Info:

Name: Nasim
 Designation: Assistant Director
 Department: CSE
 Papers: 2

Teacher Info:

Name: Nasim
 Designation: Assistant Director
 Department: EEE
 Papers: 2

Officer Info:

Name: Ahdab
 Designation: Clark (Grade-1)
 Office: HR
 Association: True

Staff Info:

Name: Kuddus
 Designation: Sweeper
 Overtime: 10.75

Analysis

Teachers displayed department names correctly. However, mapping logic was duplicated inside the class.

Design Impact

Direct navigation leads to tight coupling and reduces flexibility. Global changes to department logic require editing multiple places.

Lab 6: Navigation – Nonrefactoring

Without refactoring, computation and policy logic remain tightly coupled, leading to rigidity.

Procedure

- Implemented clsSalaryCalculator using an enum and switch.
- Associated employees with salary calculations directly.

Implementation

using System;
 namespace Polymorphism

```

{
class Program
{
    public class clsDepartment
    {
        public int DepartmentID { get; set; }
        public string DepartmentName { get; set; }
    }

    public class clsSalaryCalculator
    {
        public enum AppointmentType { Permanent, Temporary, Casual }
        private double basicSalary;
        private AppointmentType type;

        public clsSalaryCalculator(double salary, AppointmentType type)
        {
            basicSalary = salary; this.type = type;
        }

        public double getTotalSalary() => type switch
        {
            AppointmentType.Permanent => basicSalary * 2,
            AppointmentType.Temporary => basicSalary * 1.5,
            AppointmentType.Casual => basicSalary * 1.1,
            _ => throw new Exception("No Salary defined")
        };
    }

    public abstract class clsEmployee
    {
        public string Name { get; set; }
        public string Designation { get; set; }
        public double BasicSalary { get; set; }
        public abstract void WriteInfo();
    }

    public class clsTeacher : clsEmployee
    {
        public int DepartmentID { get; set; }
        public int Papers { get; set; }
        public clsDepartment Department => DepartmentID switch
        {

```

```

1 => new clsDepartment { DepartmentID = 1, DepartmentName = "CSE" },
2 => new clsDepartment { DepartmentID = 2, DepartmentName = "EEE" },
_ => new clsDepartment()
};

public override void WriteInfo()
{
    Console.WriteLine("Teacher Info:");
    Console.WriteLine($"Name: {Name}\nDesignation: {Designation}\nDepartment:
{Department.DepartmentName}\nPapers: {Papers}\n");
}
}

public class clsOfficer : clsEmployee
{
    public string Office { get; set; }
    public bool AssociationMember { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Officer Info:");
        Console.WriteLine($"Name: {Name}\nDesignation: {Designation}\nOffice:
{Office}\nAssociation: {AssociationMember}\n");
    }
}

public class clsStaff : clsEmployee
{
    public double OverTime { get; set; }

    public override void WriteInfo()
    {
        Console.WriteLine("Staff Info:");
        Console.WriteLine($"Name: {Name}\nDesignation: {Designation}\nOvertime:
{OverTime}\n");
    }
}

static void Main()
{
    clsEmployee[] employees = new clsEmployee[4];
    clsSalaryCalculator[] salaries = new clsSalaryCalculator[4];

```

```

        employees[0] = new clsTeacher { Name = "Nasim", Designation = "Assistant
Professor", BasicSalary = 100000, DepartmentID = 1, Papers = 2 };
        salaries[0] = new clsSalaryCalculator(employees[0].BasicSalary,
clsSalaryCalculator.AppointmentType.Permanent);

        employees[1] = new clsTeacher { Name = "Rahman", Designation = "Lecturer",
BasicSalary = 100000, DepartmentID = 2, Papers = 2 };
        salaries[1] = new clsSalaryCalculator(employees[1].BasicSalary,
clsSalaryCalculator.AppointmentType.Temporary);

        employees[2] = new clsOfficer { Name = "Ahdab", Designation = "Clark (Grade-1)",
BasicSalary = 100000, Office = "HR", AssociationMember = true };
        salaries[2] = new clsSalaryCalculator(employees[2].BasicSalary,
clsSalaryCalculator.AppointmentType.Casual);

        employees[3] = new clsStaff { Name = "Kuddus", Designation = "Sweeper",
BasicSalary = 50000, OverTime = 10.75 };
        salaries[3] = new clsSalaryCalculator(employees[3].BasicSalary,
clsSalaryCalculator.AppointmentType.Permanent);

        for (int i = 0; i < 4; i++)
        {
            employees[i].WriteInfo();
            Console.WriteLine("Total Salary: " + salaries[i].getTotalSalary() + "\n");
        }

        Console.ReadLine();
    }
}

```

Result/Output:

Teacher Info:

Name: Nasim

Designation: Assistant Professor

Department: CSE

Papers: 2

Total Salary: 200000

Teacher Info:

Name: Rahman

Designation: Lecturer

Department: EEE

Papers: 2
Total Salary: 150000

Officer Info:
Name: Ahdab
Designation: Clark (Grade-1)
Office: HR
Association: True
Total Salary: 110000

Staff Info:
Name: Kuddus
Designation: Sweeper
Overtime: 10.75
Total Salary: 100000

Analysis

Correct salary totals were computed, but the calculator handled both computation and business rules together.

Design Impact

This design is fragile. Policy changes require modifying the calculator, violating the Open/Closed Principle.

Task 6 : Navigation – Refactoring

Refactoring separates variable policies into dedicated classes, improving modularity and extensibility.

Procedure

- Created subclasses of clsAppointmentType (Permanent, Temporary, Casual) to encapsulate salary bonus logic.
- Modified clsSalaryCalculator to delegate to appointment type objects.

Implementation

```
using System;
namespace Polymorphism
{
    class Program
    {
        public class clsDepartment
        {
```



```

    public int DepartmentID { get; set; }
    public string DepartmentName { get; set; }
}

public abstract class clsAppointmentType
{
    protected double bonusPercent = 0;
    public virtual double getBonusPercent() => bonusPercent;
}

public class clsPermanent : clsAppointmentType { public clsPermanent() =>
bonusPercent = 1.0; }
public class clsTemporary : clsAppointmentType { public clsTemporary() =>
bonusPercent = 0.5; }
public class clsCasual : clsAppointmentType { public clsCasual() => bonusPercent =
0.1; }

public class clsSalaryCalculator
{
    private double basicSalary;
    private clsAppointmentType appointmentType;
    public clsSalaryCalculator(double salary, clsAppointmentType type)
    {
        basicSalary = salary; appointmentType = type;
    }
    public double getTotalSalary() => basicSalary + basicSalary *
appointmentType.getBonusPercent();
}

public abstract class clsEmployee
{
    public string Name { get; set; }
    public string Designation { get; set; }
    public double BasicSalary { get; set; }
    public abstract void WriteInfo();
}

public class clsTeacher : clsEmployee
{
    public int DepartmentID { get; set; }
    public int Papers { get; set; }
    public clsDepartment Department => DepartmentID switch
    {

```

```

        1 => new clsDepartment { DepartmentID = 1, DepartmentName = "CSE" },
        2 => new clsDepartment { DepartmentID = 2, DepartmentName = "EEE" },
        _ => new clsDepartment()
    };
    public override void WriteInfo()
    {
        Console.WriteLine($"Teacher Info:\nName: {Name}\nDesignation:
{Designation}\nDepartment: {Department.DepartmentName}\nPapers: {Papers}\n");
    }
}

public class clsOfficer : clsEmployee
{
    public string Office { get; set; }
    public bool AssociationMember { get; set; }
    public override void WriteInfo()
    {
        Console.WriteLine($"Officer Info:\nName: {Name}\nDesignation:
{Designation}\nOffice: {Office}\nAssociation: {AssociationMember}\n");
    }
}

public class clsStaff : clsEmployee
{
    public double OverTime { get; set; }
    public override void WriteInfo()
    {
        Console.WriteLine($"Staff Info:\nName: {Name}\nDesignation:
{Designation}\nOvertime: {OverTime}\n");
    }
}

static void Main()
{
    clsEmployee[] employees = new clsEmployee[4];
    clsSalaryCalculator[] salaries = new clsSalaryCalculator[4];

    employees[0] = new clsTeacher { Name = "Nasim", Designation = "Assistant
Professor", BasicSalary = 100000, DepartmentID = 1, Papers = 2 };
    salaries[0] = new clsSalaryCalculator(employees[0].BasicSalary, new
clsPermanent());

```

```

        employees[1] = new clsTeacher { Name = "Rahman", Designation = "Lecturer",
BasicSalary = 100000, DepartmentID = 2, Papers = 2 };
        salaries[1] = new clsSalaryCalculator(employees[1].BasicSalary, new
clsTemporary());

        employees[2] = new clsOfficer { Name = "Ahdab", Designation = "Clark (Grade-1)",
BasicSalary = 100000, Office = "HR", AssociationMember = true };
        salaries[2] = new clsSalaryCalculator(employees[2].BasicSalary, new clsCasual());

        employees[3] = new clsStaff { Name = "Kuddus", Designation = "Sweeper",
BasicSalary = 50000, OverTime = 10.75 };
        salaries[3] = new clsSalaryCalculator(employees[3].BasicSalary, new
clsPermanent());

        for (int i = 0; i < 4; i++)
        {
            employees[i].WriteInfo();
            Console.WriteLine("Total Salary: " + salaries[i].getTotalSalary() + "\n");
        }

        Console.ReadLine();
    }
}

```

Result/Output:

Teacher Info:

Name: Nasim

Designation: Assistant Professor

Department: CSE

Papers: 2

Total Salary: 200000

Teacher Info:

Name: Rahman

Designation: Lecturer

Department: EEE

Papers: 2

Total Salary: 150000

Officer Info:

Name: Ahdab

Designation: Clark (Grade-1)

Office: HR
 Association: True
 Total Salary: 110000

Staff Info:
 Name: Kuddus
 Designation: Sweeper
 Overtime: 10.75
 Total Salary: 100000

Analysis

Same results as Task 6, but with cleaner, more modular code.

Design Impact

The system now adheres to Separation of Concerns. Adding new appointment types requires no modification of calculator logic, making the system extensible.

Task 8: Adapter Pattern

The Adapter pattern bridges incompatible interfaces, enabling reuse of legacy or third-party systems without modification.

Procedure

- Defined IEmployeeAdapter as a standard interface.
- Implemented adapters for teachers and officers to translate legacy string arrays into domain objects.

Implementation

```
using System;
using System.Collections.Generic;
```

```
namespace Polymorphism
{
    class Program
    {
        // Singleton University
        public sealed class University
        {
            private static University instance = null;
            private string Name = "Jashore University of Science and Technology";
            private string Address = "Ambottola, Jashore";

            public static University getUniversityInstance()
```

```

    {
        if (instance == null) instance = new University();
        return instance;
    }

    public void getUniversityInfo() => Console.WriteLine($"-----\n{Name},
{Address}\n-----");
}

// Department DB
public sealed class clsDepartmentDB
{
    private static List<clsDepartment> Departments = new List<clsDepartment>
    {
        new clsDepartment(1,"CSE"),
        new clsDepartment(2,"EEE")
    };

    public clsDepartment getDepartment(int id)
    {
        foreach(var d in Departments) if(d.DepartmentID==id) return d;
        return new clsDepartment(0,"Department Name Not Found");
    }
}

public class clsDepartment { public int DepartmentID; public string DepartmentName;
public clsDepartment(){} public clsDepartment(int id,string
name){DepartmentID=id;DepartmentName=name;} }

// Abstract Employee
public abstract class clsEmployee
{
    public string Name, ParmanentAddress, CurrentAddress, Phone, Designation;
    public DateTime DateofBirth;
    public char Sex='M';
    public abstract void WriteInfo();
}

// Teacher
public class clsTeacher : clsEmployee
{
    public int DepartmentID, Papers;
    public bool PhD = false;
}

```

```

        public clsDepartment Department => new
        clsDepartmentDB().getDepartment(DepartmentID);
        public override void WriteInfo()
        {
            University.getUniversityInstance().getUniversityInfo();
            Console.WriteLine($"{(PhD?"Dr. ":"Mr. ")}Name: {Name}\nDOB:
{DateofBirth}\nSex: {Sex}\nPermanent: {ParmanentAddress}\nCurrent:
{CurrentAddress}\nPhone: {Phone}\nDesignation: {Designation}\nDepartment:
{Department.DepartmentName}\nPapers: {Papers}\n");
        }
    }

// Officer
public class clsOfficer : clsEmployee
{
    public string Office; public bool AssociationMember;
    public override void WriteInfo()
    {
        University.getUniversityInstance().getUniversityInfo();
        Console.WriteLine($"Name: {Name}\nDOB: {DateofBirth}\nSex:
{Sex}\nPermanent: {ParmanentAddress}\nCurrent: {CurrentAddress}\nPhone:
{Phone}\nDesignation: {Designation}\nOffice: {Office}\nAssociation:
{AssociationMember}\n");
    }
}

// Staff
public class clsStaff : clsEmployee
{
    public double OverTime;
    public override void WriteInfo()
    {
        University.getUniversityInstance().getUniversityInfo();
        Console.WriteLine($"Name: {Name}\nDOB: {DateofBirth}\nSex:
{Sex}\nPermanent: {ParmanentAddress}\nCurrent: {CurrentAddress}\nPhone:
{Phone}\nDesignation: {Designation}\nOvertime: {OverTime}\n");
    }
}

// Factory
public class clsFacultyFactory
{
    public clsEmployee getTeacher(string qual)

```

```

    {
        var t = new clsTeacher { Name = qual=="PhD"? "Nasim": "Imran",
DateofBirth=qual=="PhD"? Convert.ToDateTime("01/01/1979"): Convert.ToDateTime("01/0
1/1978"), Sex='M', ParmanentAddress=qual=="PhD"? "Jhenaidah": "Jashore",
CurrentAddress="Mirpur", Phone="01730016854", Designation="Assistant Professor",
DepartmentID=qual=="PhD"? 1:2, Papers=qual=="PhD"? 20:2, PhD=qual=="PhD"};
        return t;
    }
}

```

// Adapter Interface

```
public interface IEmployeeAdapter { List<clsEmployee> getEmployees(); }
```

```

public class clsLegacyTeachingHRSysytem { public string[][] getTeachers() => new
string[][] { new[] { "PhD", "Arif", "01/01/1979", "Jashore", "Assistant Professor", "CSE" },
new[] { "MSc", "Rakib", "01/01/1982", "Khulna", "Lecturer", "EEE" } }; }

```

```

public class clsLegacyOfficialHRSysytem { public string[][] getOfficers() => new
string[][] { new[] { "Reza", "01/01/1979", "Jashore", "Clark (Grade-2)", "HR" },
new[] { "Hakim", "01/01/1982", "Khulna", "Clark (Grade-3)", "ACC" } }; }

```

```

public class clsTeacherAdapter : IEmployeeAdapter
{
    public List<clsEmployee> getEmployees()
    {
        var oEmployees = new List<clsEmployee>();
        var legacy = new clsLegacyTeachingHRSysytem();
        var factory = new clsFacultyFactory();
        foreach(var e in legacy.getTeachers()) oEmployees.Add(factory.getTeacher(e[0]));
        return oEmployees;
    }
}

```

```

public class clsOfficerAdapter : IEmployeeAdapter
{
    public List<clsEmployee> getEmployees()
    {
        var oEmployees = new List<clsEmployee>();
        var legacy = new clsLegacyOfficialHRSysytem();
        foreach(var e in legacy.getOfficers())
        {
            oEmployees.Add(new clsOfficer { Name=e[0],
DateofBirth=Convert.ToDateTime(e[1]), Sex='M', ParmanentAddress=e[2],

```

```

CurrentAddress="NA", Phone="NA", Designation=e[3], Office=e[4],
AssociationMember=true });
    }
    return oEmployees;
}
}

static void Main()
{
    clsEmployee[] oEmployees = new clsEmployee[4];
    var factory = new clsFacultyFactory();
    oEmployees[0]=factory.getTeacher("PhD");
    oEmployees[1]=factory.getTeacher("MSc");
    oEmployees[2]=new clsOfficer { Name="Ahdab",
DateofBirth=Convert.ToDateTime("01/01/1975"), Sex='M', ParmanentAddress="Dinajpur",
CurrentAddress="Kalyanpur", Phone="01712345678", Designation="Clark (Grade-1)",
Office="HR", AssociationMember=true };
    oEmployees[3]=new clsStaff { Name="Kuddus",
DateofBirth=Convert.ToDateTime("01/01/1980"), Sex='M', ParmanentAddress="Dinajpur",
CurrentAddress="Kalyanpur", Phone="01712345678", Designation="Sweeper",
OverTime=10.75 };

    foreach(var e in oEmployees) { e.WriteInfo(); Console.WriteLine(); }

    IEmployeeAdapter adapter = new clsTeacherAdapter();
    Console.WriteLine("##### Teacher List from Legacy System #####");
    foreach(var e in adapter.getEmployees()) e.WriteInfo();

    adapter = new clsOfficerAdapter();
    Console.WriteLine("##### Officer List from Legacy System #####");
    foreach(var e in adapter.getEmployees()) e.WriteInfo();

    Console.ReadLine();
}
}
}

```

Result/Output:

Jashore University of Science and Technology, Ambottola, Jashore

Name: Dr. Nasim

DOB: 01/01/1979 00:00:00

Sex: M
Permanent: Jhenaidah
Current: Mirpur
Phone: 01730016854
Designation: Assistant Professor
Department: CSE
Papers: 20

Jashore University of Science and Technology, Ambottola, Jashore

Name: Mr. Imran
DOB: 01/01/1978 00:00:00
Sex: M
Permanent: Jashore
Current: Mirpur
Phone: 01730016854
Designation: Assistant Professor
Department: EEE
Papers: 2

Jashore University of Science and Technology, Ambottola, Jashore

Name: Ahdab
DOB: 01/01/1975 00:00:00
Sex: M
Permanent: Dinajpur
Current: Kalyanpur
Phone: 01712345678
Designation: Clark (Grade-1)
Office: HR
Association: True

Jashore University of Science and Technology, Ambottola, Jashore

Name: Kuddus
DOB: 01/01/1980 00:00:00
Sex: M

Permanent: Dinajpur
 Current: Kalyanpur
 Phone: 01712345678
 Designation: Sweeper
 Overtime: 10.75

Teacher List from Legacy System

 Jashore University of Science and Technology, Ambottola, Jashore

Name: Dr. Nasim
 DOB: 01/01/1979 00:00:00
 Sex: M
 Permanent: Jhenaidah
 Current: Mirpur
 Phone: 01730016854
 Designation: Assistant Professor
 Department: CSE
 Papers: 20

 Jashore University of Science and Technology, Ambottola, Jashore

Name: Mr. Imran
 DOB: 01/01/1978 00:00:00
 Sex: M
 Permanent: Jashore
 Current: Mirpur
 Phone: 01730016854
 Designation: Assistant Professor
 Department: EEE
 Papers: 2

Officer List from Legacy System

 Jashore University of Science and Technology, Ambottola, Jashore

Name: Reza
 DOB: 01/01/1979 00:00:00
 Sex: M

Permanent: Jashore
 Current: NA
 Phone: NA
 Designation: Clark (Grade-2)
 Office: HR
 Association: True

 Jashore University of Science and Technology, Ambottola, Jashore

Name: Hakim
 DOB: 01/01/1982 00:00:00
 Sex: M
 Permanent: Khulna
 Current: NA
 Phone: NA
 Designation: Clark (Grade-3)
 Office: ACC
 Association: True

Analysis

Legacy datasets were converted and displayed as structured employee objects.

Design Impact

Adapters isolate compatibility issues, making the system robust to legacy integration and enhancing reusability.

Task 9: Builder Pattern

The Builder pattern separates construction from representation, allowing step-by-step creation of complex objects.

Procedure

- Implemented role-specific builder-style classes for teachers, officers, and staff.
- Used FacultyFactory to streamline construction.

Implementation

```
using System;
using System.Collections.Generic;
```

```
namespace Polymorphism
{
    class Program
```

```

{
// ----- Singleton University -----
public sealed class University
{
    private static University instance = null;
    private string name = "Jashore University of Science and Technology";
    private string address = "Ambottola, Jashore";
    public static University getInstance()
    {
        if (instance == null) instance = new University();
        return instance;
    }
    public void PrintInfo() => Console.WriteLine($"-----\n{name},
{address}\n-----");
}

// ----- Department -----
public class Department { public int ID; public string Name; public Department(int id,
string name) { ID = id; Name = name; } }
public sealed class DepartmentDB
{
    private static List<Department> depts = new() { new Department(1, "CSE"), new
Department(2, "EEE") };
    public Department GetDept(int id) => depts.Find(d => d.ID == id) ?? new
Department(0, "Department Not Found");
}

// ----- Abstract Employee -----
public abstract class Employee
{
    public string Name; public DateTime DOB; public char Sex; public string PAddress;
public string CAddress;
    public string Phone; public string Designation;
    public abstract void WriteInfo();
}

// ----- Teachers -----
public class TeacherPhD : Employee
{
    public int DeptID, Papers;
    public Department Department => new DepartmentDB().GetDept(DeptID);
    public override void WriteInfo()
    {

```

```

        University.getInstance().PrintInfo();
        Console.WriteLine($"Teacher Info:\n-----\nName: Dr. {Name}\nDOB:
{DOB}\nSex: {Sex}\nPermanent: {PAddress}\nCurrent: {CAddress}\nPhone:
{Phone}\nDesignation: {Designation}\nDepartment: {Department.Name}\nPapers:
{Papers}\n");
    }
}

```

```

public class TeacherNonPhD : Employee
{
    public int DeptID, Papers;
    public Department Department => new DepartmentDB().GetDept(DeptID);
    public override void WriteInfo()
    {
        University.getInstance().PrintInfo();
        Console.WriteLine($"Teacher Info:\n-----\nName: Mr. {Name}\nDOB:
{DOB}\nSex: {Sex}\nPermanent: {PAddress}\nCurrent: {CAddress}\nPhone:
{Phone}\nDesignation: {Designation}\nDepartment: {Department.Name}\nPapers:
{Papers}\n");
    }
}

```

```

// ----- Officer & Staff -----
public class Officer : Employee { public string Office; public bool Assoc; public
override void WriteInfo() { University.getInstance().PrintInfo();
Console.WriteLine($"Officer Info:\n-----\nName: {Name}\nDOB: {DOB}\nSex:
{Sex}\nPermanent: {PAddress}\nCurrent: {CAddress}\nPhone: {Phone}\nDesignation:
{Designation}\nOffice: {Office}\nAssociation: {Assoc}\n"); } }
public class Staff : Employee { public double OverTime; public override void
WriteInfo() { University.getInstance().PrintInfo(); Console.WriteLine($"Staff Info:\n-----
-----\nName: {Name}\nDOB: {DOB}\nSex: {Sex}\nPermanent: {PAddress}\nCurrent:
{CAddress}\nPhone: {Phone}\nDesignation: {Designation}\nOvertime: {OverTime}\n"); } }

```

```

// ----- Factory -----
public class FacultyFactory
{
    public Employee GetTeacher(string qual)
    {
        if (qual == "PhD") return new TeacherPhD { Name = "Nasim", DOB =
DateTime.Parse("01/01/1979"), Sex = 'M', PAddress = "Jhenaidah", CAddress = "Mirpur",
Phone = "01730016854", Designation = "Assistant Professor", DeptID = 1, Papers = 20 };
    }
}

```

```

        if (qual == "MSc") return new TeacherNonPhD { Name = "Imran", DOB =
DateTime.Parse("01/01/1978"), Sex = 'M', PAddress = "Jashore", CAddress = "Bosila",
Phone = "01730016854", Designation = "Assistant Professor", DeptID = 2, Papers = 2 };
        return null;
    }
    public Employee GetTeacher(string qual, string name, string dob, string pAddr, string
desig, string dept)
    {
        int deptID = dept == "CSE" ? 1 : 2;
        if (qual == "PhD") return new TeacherPhD { Name = name, DOB =
DateTime.Parse(dob), PAddress = pAddr, CAddress = "NA", Designation = desig, DeptID =
deptID };
        if (qual == "MSc") return new TeacherNonPhD { Name = name, DOB =
DateTime.Parse(dob), PAddress = pAddr, CAddress = "NA", Designation = desig, DeptID =
deptID };
        return null;
    }
}

```

// ----- Adapter -----

```

public interface IAdapter { List<Employee> GetEmployees(); }

public class LegacyTeacherHR { public string[][] Get() => new[] { new[] { "PhD",
"Arif", "01/01/1979", "Jashore", "Assistant Professor", "CSE" }, new[] { "MSc", "Rakib",
"01/01/1982", "Khulna", "Lecturer", "EEE" } }; }

public class LegacyOfficerHR { public string[][] Get() => new[] { new[] { "Reza",
"01/01/1979", "Jashore", "Clark (Grade-2)", "HR" }, new[] { "Hakim", "01/01/1982",
"Khulna", "Clark (Grade-3)", "ACC" } }; }

```

```

public class TeacherAdapter : IAdapter
{
    public List<Employee> GetEmployees()
    {
        List<Employee> list = new();
        var legacy = new LegacyTeacherHR().Get();
        foreach (var t in legacy) list.Add(new FacultyFactory().GetTeacher(t[0], t[1], t[2],
t[3], t[4], t[5]));
        return list;
    }
}

```

```

public class OfficerAdapter : IAdapter
{

```

```

public List<Employee> GetEmployees()
{
    List<Employee> list = new();
    var legacy = new LegacyOfficerHR().Get();
    foreach (var o in legacy) list.Add(new Officer { Name = o[0], DOB =
DateTime.Parse(o[1]), PAddress = o[2], CAddress = "NA", Designation = o[3], Office =
o[4], Assoc = true });
    return list;
}
}

// ----- Main -----
static void Main()
{
    Employee[] emps = new Employee[4];
    var factory = new FacultyFactory();
    emps[0] = factory.GetTeacher("PhD");
    emps[1] = factory.GetTeacher("MSc");
    emps[2] = new Officer { Name = "Ahdab", DOB = DateTime.Parse("01/01/1975"),
PAddress = "Dinajpur", CAddress = "Kalyanpur", Phone = "01712345678", Designation =
"Clark (Grade-1)", Office = "HR", Assoc = true };
    emps[3] = new Staff { Name = "Kuddus", DOB = DateTime.Parse("01/01/1980"),
PAddress = "Dinajpur", CAddress = "Kalyanpur", Phone = "01712345678", Designation =
"Sweeper", OverTime = 10.75 };

    foreach (var e in emps) { e.WriteInfo(); Console.WriteLine(); }

    IAdapter adapter = new TeacherAdapter();
    Console.WriteLine("##### Teacher List from Legacy System #####");
    foreach (var e in adapter.GetEmployees()) e.WriteInfo();

    adapter = new OfficerAdapter();
    Console.WriteLine("##### Officer List from Legacy System #####");
    foreach (var e in adapter.GetEmployees()) e.WriteInfo();

    Console.ReadLine();
}
}
}

```

Output:

Jashore University of Science and Technology, Ambottola, Jashore

Teacher Info:

Name: Dr. Nasim
DOB: 1/1/1979 12:00:00 AM
Sex: M
Permanent: Jhenaidah
Current: Mirpur
Phone: 01730016854
Designation: Assistant Professor
Department: CSE
Papers: 20

Jashore University of Science and Technology, Ambottola, Jashore

Teacher Info:

Name: Mr. Imran
DOB: 1/1/1978 12:00:00 AM
Sex: M
Permanent: Jashore
Current: Bosila
Phone: 01730016854
Designation: Assistant Professor
Department: EEE
Papers: 2

Jashore University of Science and Technology, Ambottola, Jashore

Officer Info:

Name: Ahdab
DOB: 1/1/1975 12:00:00 AM
Sex: M
Permanent: Dinajpur
Current: Kalyanpur
Phone: 01712345678
Designation: Clark (Grade-1)
Office: HR

Association: True

Jashore University of Science and Technology, Ambottola, Jashore

Staff Info:

Name: Kuddus
DOB: 1/1/1980 12:00:00 AM
Sex: M
Permanent: Dinajpur
Current: Kalyanpur
Phone: 01712345678
Designation: Sweeper
Overtime: 10.75

Teacher List from Legacy System

Jashore University of Science and Technology, Ambottola, Jashore

Teacher Info:

Name: Dr. Arif
DOB: 1/1/1979 12:00:00 AM
Sex: M
Permanent: Jashore
Current: NA
Phone:
Designation: Assistant Professor
Department: CSE
Papers: 0

Jashore University of Science and Technology, Ambottola, Jashore

Teacher Info:

Name: Mr. Rakib
DOB: 1/1/1982 12:00:00 AM
Sex: M

Permanent: Khulna
 Current: NA
 Phone:
 Designation: Lecturer
 Department: EEE
 Papers: 0

Officer List from Legacy System

 Jashore University of Science and Technology, Ambottola, Jashore

Officer Info:

Name: Reza
 DOB: 1/1/1979 12:00:00 AM
 Sex: M
 Permanent: Jashore
 Current: NA
 Phone:
 Designation: Clark (Grade-2)
 Office: HR
 Association: True

 Jashore University of Science and Technology, Ambottola, Jashore

Officer Info:

Name: Hakim
 DOB: 1/1/1982 12:00:00 AM
 Sex: M
 Permanent: Khulna
 Current: NA
 Phone:
 Designation: Clark (Grade-3)
 Office: ACC
 Association: True

Results/Analysis

Objects were created consistently with structured output, regardless of complexity.

Design Impact

The Builder pattern supports flexibility in object creation, improves readability, and simplifies maintaining or extending object construction logic.

5. Conclusion

The lab combined UML modeling with OOP implementation, strengthening understanding of system design and coding practices. It highlighted the value of diagrams for clarity, OOP for modularity, and design patterns for building scalable, reusable software.